Homework 2

In this homework, we will go through the Hough transform. *This homework was adapted from Stanford CS131.*

Setup

Step 1

First, run the cells below to clone the `Emory-Computer-Vision-Course` repo and `cd` into the correct directory in order to access some necessary files.

In [26]:
```
'''
import os

if not os.path.exists("Emory-Computer-Vision-Course"):
    # Clone the repository if it doesn't already exist
    !git clone https://github.com/YoungjoongEmory/Emory-Computer-Vision-Course.git

%cd Emory-Computer-Vision-Course/Assignments/HW2/
'''
```

Out[26]:
```
'\nimport os\n\nif not os.path.exists("Emory-Computer-Vision-Course"):\n    # Clone the repository if it doesn\'t already exist\n    !git clone https://github.com/YoungjoongEmory/Emory-Computer-Vision-
Course.git\n\n%cd Emory-Computer-Vision-Course/Assignments/HW2/\n'
```

Step 2

Next, run the cells below to install the necessary libraries and packages.

In [27]:
```
# Install the necessary dependencies
# (restart your runtime session if prompted to, and then re-run this cell)

#!pip install -r requirements.txt
```

In [28]:
```
# Imports the print function from newer versions of python
from __future__ import print_function

# Numpy is the main package for scientific computing with Python.
# This will be one of our most used libraries in this class
import numpy as np

# The Time library helps us time code runtimes
import time

# PIL (Pillow) is a useful library for opening, manipulating, and saving images
from PIL import Image

# skimage (Scikit-Image) is a library for image processing
from skimage import io
from skimage import filters
from skimage.feature import corner_peaks
from skimage.io import imread

from scipy.ndimage import convolve
from scipy.spatial.distance import cdist

import cv2 as cv

# Matplotlib is a useful plotting library for python
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
# This code is to make matplotlib figures appear inline in the
# notebook rather than in a new window.
%matplotlib inline
plt.rcParams['figure.figsize'] = (15.0, 12.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'
```

Part 1: Lane Detection (15 points)

In this section we will implement a simple lane detection application using Canny edge detector and Hough transform.

The algorithm can broken down into the following steps:

1. Detect edges using the Canny edge detector.
2. Extract the edges in the region of interest (a triangle covering the bottom corners and the center of the image).
3. Run Hough transform to detect lanes.

1.1 Edge detection

Lanes on the roads are usually thin and long lines with bright colors. Our edge detection algorithm by itself should be able to find the lanes pretty well. Run the code cell below to load the example image and detect edges from the image.

Add your code from HW1 for the functions below.

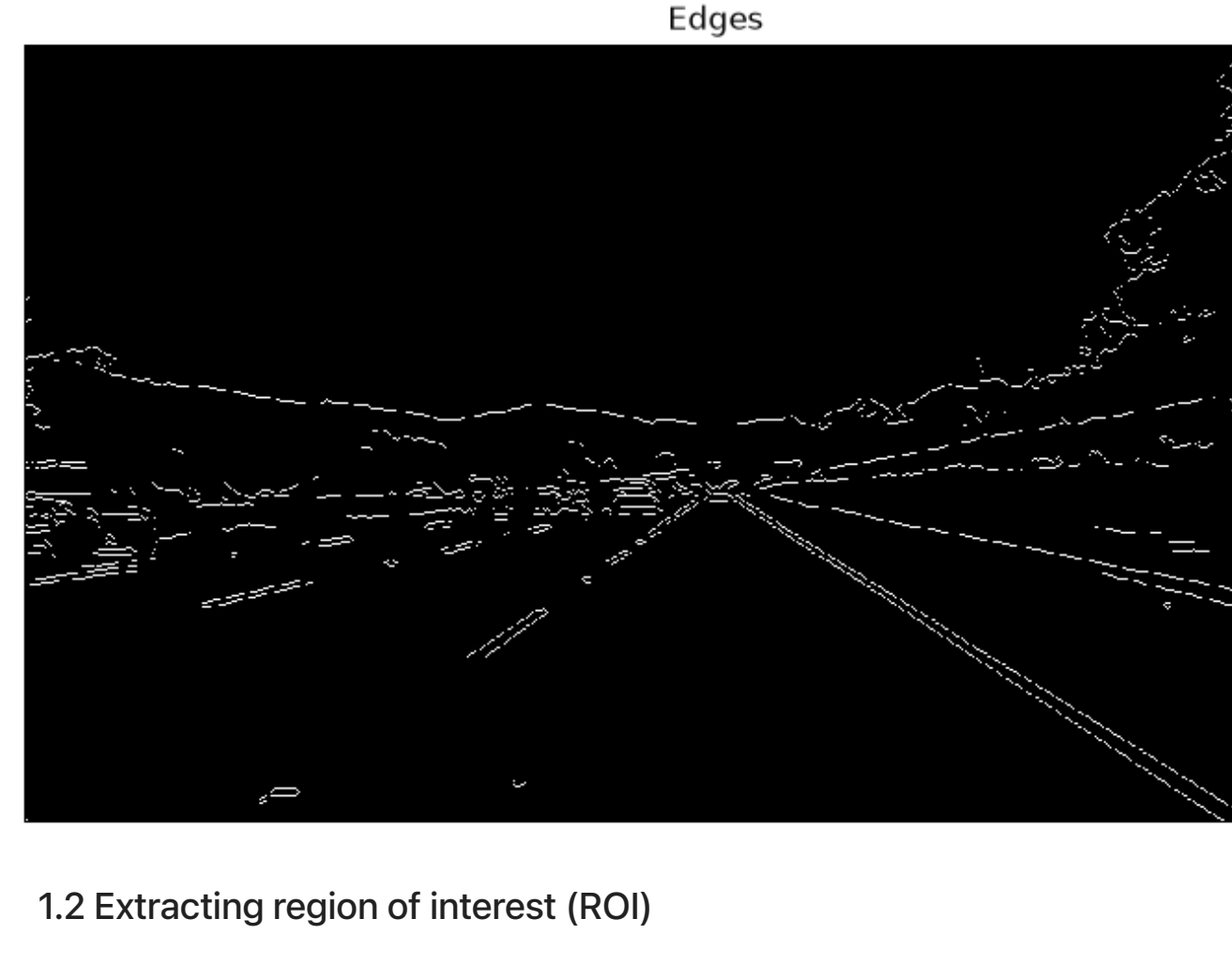
In [29]:
```
def gaussian_kernel(size, sigma):
    """ Implementation of Gaussian Kernel.

    This function follows the gaussian kernel formula,
    and creates a kernel matrix.

    Hints:
    - Use np.pi and np.exp to compute pi and exp.

    Args:
        size: int of the size of output matrix.
        sigma: float of sigma to calculate kernel.

    Returns:
        kernel: numpy array of shape (size, size).
    """

    kernel = np.zeros((size, size))

    ### YOUR CODE HERE
    denom = 1/(2*np.pi*np.square(sigma))
    k = (size - 1)//2
    for i in range(size):
        for j in range(size):
            kernel[i, j] = denom * np.exp((np.square(i-k) + np.square(j-k))/(-2*np.square(sigma)))

    ### END YOUR CODE

    return kernel

def conv(image, kernel):
    """ An implementation of convolution filter.

    This function uses element-wise multiplication and np.sum()
    to efficiently compute weighted sum of neighborhood at each
    pixel.

    Args:
        image: numpy array of shape (Hi, Wi).
        kernel: numpy array of shape (Hk, Wk).

    Returns:
        out: numpy array of shape (Hi, Wi).
    """
    Hi, Wi = image.shape
    Hk, Wk = kernel.shape
    out = np.zeros((Hi, Wi))

    # For this assignment, we will use edge values to pad the images.
    # Zero padding will make derivatives at the image boundary very big,
    # whereas we want to ignore the edges at the boundary.
    pad_width0 = Hk // 2
    pad_width1 = Wk // 2
    pad_width = ((pad_width0,pad_width0),(pad_width1,pad_width1))
    padded = np.pad(image, pad_width, mode='edge')

    ### YOUR CODE HERE
    k_f = np.flip(kernel).flatten()
    for r in range(Hi):
        b = r + 2*pad_width0 + 1
        for c in range(Wi):
            area = padded[r:b, c:c+1+2*pad_width1].flatten()
            out[r, c] = np.dot(area, k_f)


    ### END YOUR CODE

    return out

def partial_x(img):
    """ Computes partial x-derivative of input img.

    Hints:
        - You may use the conv function in defined in this file.

    Args:
        img: numpy array of shape (H, W).
    Returns:
        out: x-derivative image.
    """

    out = None

    ### YOUR CODE HERE
    d_x = np.array([[0.5, 0, -0.5]])

    out = conv(img, d_x)

    ### END YOUR CODE

    return out

def partial_y(img):
    """ Computes partial y-derivative of input img.

    Hints:
        - You may use the conv function in defined in this file.

    Args:
        img: numpy array of shape (H, W).
    Returns:
        out: y-derivative image.
    """

    out = None

    ### YOUR CODE HERE
    d_y = np.transpose(np.array([[0.5, 0, -0.5]]))
    out = conv(img, d_y)

    ### END YOUR CODE

    return out

def gradient(img):
    """ Returns gradient magnitude and direction of input img.

    Args:
        img: Grayscale image. Numpy array of shape (H, W).

    Returns:
        G: Magnitude of gradient at each pixel in img.
            Numpy array of shape (H, W).
        theta: Direction(in degrees, 0 <= theta < 360) of gradient
            at each pixel in img. Numpy array of shape (H, W).

    Hints:
        - Use np.sqrt and np.arctan2 to calculate square root and arctan
    """
    G = np.zeros(img.shape)
    theta = np.zeros(img.shape)


    ### YOUR CODE HERE
    G_x = partial_x(img)
    G_y = partial_y(img)
    H = img.shape[0]
    W = img.shape[1]

    for r in range(H):
        for c in range(W):
            G[r,c] = np.sqrt(np.square(G_x[r,c]) + np.square(G_y[r,c]))
            theta[r, c] = np.arctan2(G_x[r,c], G_y[r,c])


    ### END YOUR CODE

    return G, theta

def non_maximum_suppression(G, theta):
    """ Performs non-maximum suppression.

    This function performs non-maximum suppression along the direction
    of gradient (theta) on the gradient magnitude image (G).

    Args:
        G: gradient magnitude image with shape of (H, W).
        theta: direction of gradients with shape of (H, W).

    Returns:
        out: non-maxima suppressed image.
    """
    H, W = G.shape
    out = np.zeros((H, W))

    # Round the gradient direction to the nearest 45 degrees
    theta = np.floor((theta + 22.5) / 45) * 45
    theta = (theta % 360.0).astype(np.int32)

    #print(G)

    ### BEGIN YOUR CODE
    n_dir = {
        0: (0, 1),
        1: (-1, 1),
        2: (-1, 0),
        3: (-1,-1)
    }
    for r in range(1,H-1):
        for c in range(1, W-1):
            g_d = theta[r,c]
            cate = g_d / 45 % 4
            op = n_dir[cate]
            n1 = (r + op[1], c + op[0])
            n2 = (r - op[1], c - op[0])
            if G[n1[0], n1[1]] > G[r, c] or G[n2[0], n2[1]] > G[r, c]:
                out[r, c] = 0
            else:
                out[r, c] = G[r, c]

    ### END YOUR CODE

    return out

def double_thresholding(img, high, low):
    """
    Args:
        img: numpy array of shape (H, W) representing NMS edge response.
        high: high threshold(float) for strong edges.
        low: low threshold(float) for weak edges.

    Returns:
        strong_edges: Boolean array representing strong edges.
            Strong edeges are the pixels with the values greater than
            the higher threshold.
        weak_edges: Boolean array representing weak edges.
            Weak edges are the pixels with the values smaller or equal to the
            higher threshold and greater than the lower threshold.
    """

    strong_edges = np.zeros(img.shape, dtype=bool)
    weak_edges = np.zeros(img.shape, dtype=bool)

    ### YOUR CODE HERE
    H = img.shape[0]
    W = img.shape[1]

    for r in range(H):
        for c in range(W):
            if img[r, c] > high:
                strong_edges[r,c] = 1
            elif img[r, c] > low:
                weak_edges[r,c] = 1

    ### END YOUR CODE

    return strong_edges, weak_edges

def link_edges(strong_edges, weak_edges):
    """ Find weak edges connected to strong edges and link them.

    Iterate over each pixel in strong_edges and perform breadth first
    search across the connected pixels in weak_edges to link them.
    Here we consider a pixel (a, b) is connected to a pixel (c, d)
    if (a, b) is one of the eight neighboring pixels of (c, d).

    Args:
        strong_edges: binary image of shape (H, W).
        weak_edges: binary image of shape (H, W).

    Returns:
        edges: numpy boolean array of shape(H, W).
    """

    H, W = strong_edges.shape
    indices = np.stack(np.nonzero(strong_edges)).T
    edges = np.zeros((H, W), dtype=bool)

    # Make new instances of arguments to leave the original
    # references intact
    weak_edges = np.copy(weak_edges)
    edges = np.copy(strong_edges)

    ### YOUR CODE HERE
    queue = [(r[0],r[1]) for r in indices]
    while queue:
        r, c = queue.pop(0)
        for n in get_neighbors(r, c, H, W):
            n_r, n_c = n[0], n[1]
            if weak_edges[n_r, n_c] and not edges[n_r, n_c]:
                queue.append((n_r, n_c))
                edges[n_r, n_c] = 1

    ### END YOUR CODE

    return edges

def get_neighbors(y, x, H, W):
    """ Return indices of valid neighbors of (y, x).

    Return indices of all the valid neighbors of (y, x) in an array of
    shape (H, W). An index (i, j) of a valid neighbor should satisfy
    the following:
        1. i >= 0 and i < H
        2. j >= 0 and j < W
        3. (i, j) != (y, x)

    Args:
        y, x: location of the pixel.
        H, W: size of the image.
    Returns:
        neighbors: list of indices of neighboring pixels [(i, j)].
    """
    neighbors = []

    for i in (y-1, y, y+1):
        for j in (x-1, x, x+1):
            if i >= 0 and i < H and j >= 0 and j < W:
                if (i == y and j == x):
                    continue
                neighbors.append((i, j))

    return neighbors
```

In [30]:
```
def canny(img, kernel_size=5, sigma=1.4, high=20, low=15):
    """ Implement canny edge detector by calling functions above.

    Args:
        img: binary image of shape (H, W).
        kernel_size: int of size for kernel matrix.
        sigma: float for calculating kernel.
        high: high threshold for strong edges.
        low: low threashold for weak edges.
    Returns:
        edge: numpy array of shape(H, W).
    """
    ### YOUR CODE HERE
    # 1. smoothing
    smoothed_img = conv(img, gaussian_kernel(kernel_size, sigma))

    # 2. Finding gradients
    gra, dir = gradient(smoothed_img)

    # 3. Non-maximum suppression
    sup_img = non_maximum_suppression(gra, dir)

    # 4. Double thresholding
    s_edge, w_edge = double_thresholding(sup_img, high, low)

    # 5. Edge tracking

    edge = link_edges(s_edge, w_edge)

    ### END YOUR CODE

    return edge
```

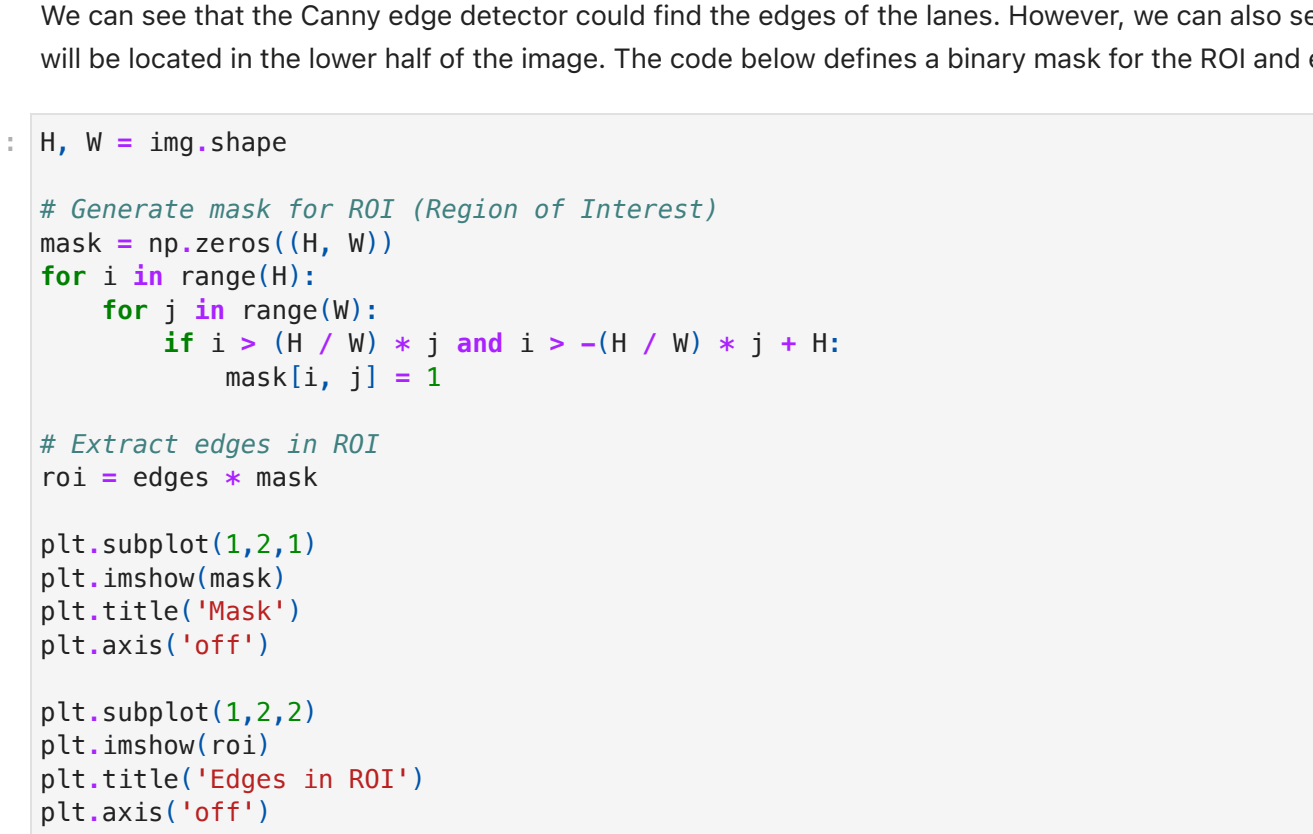
In [31]:
```
# Load image
img = io.imread('road.jpg', as_gray=True)
```

In [32]:
```
# Run Canny edge detector
edges = canny(img, kernel_size=5, sigma=1.4, high=0.03, low=0.02)

plt.subplot(211)
plt.imshow(img)
plt.axis('off')
plt.title('Input Image')

plt.subplot(212)
plt.imshow(edges)
plt.axis('off')
plt.title('Edges')
plt.show()
```

Input Image



Edges

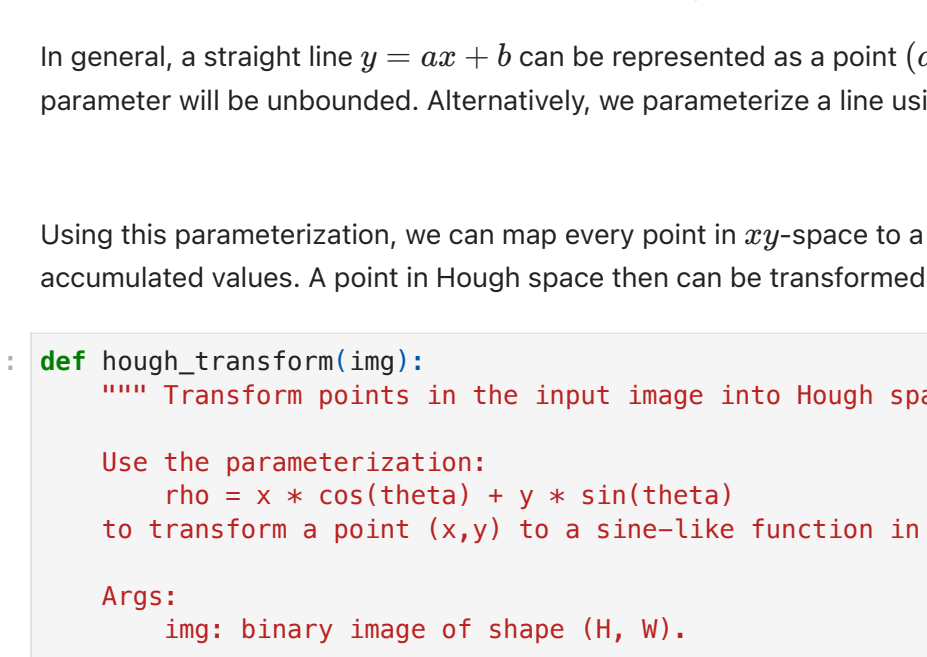


1.2 Extracting region of interest (ROI)

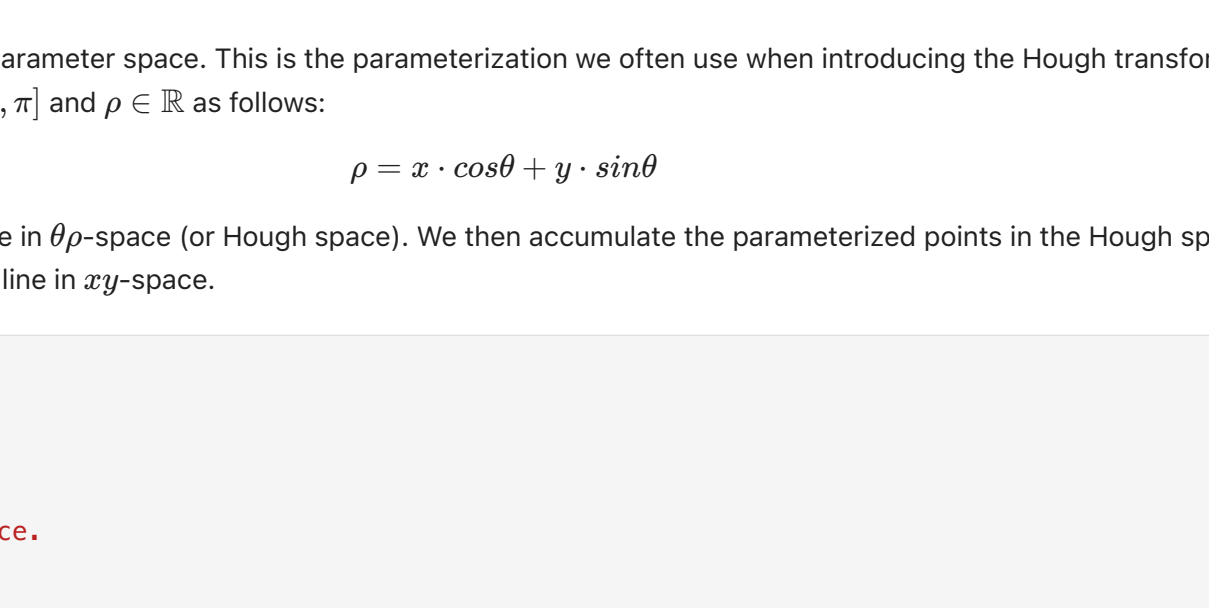
We can see that the Canny edge detector could find the edges of the lanes. However, we can also see that there are edges of other objects that we are not interested in. Given the position and orientation of the camera, we know that the lanes will be located in the lower half of the image. The code below defines a binary mask for the ROI and extract the edges within the region.

In [33]:
```
H, W = img.shape

# Generate mask for ROI (Region of Interest)
mask = np.zeros((H, W))
for i in range(H):
    for j in range(W):
        if i > (H / W) * j and i > -(H / W) * j + H:
            mask[i, j] = 1

# Extract edges in ROI
roi = edges * mask

plt.subplot(1,2,1)
plt.imshow(mask)
plt.title('Mask')
plt.axis('off')

plt.subplot(1,2,2)
plt.imshow(roi)
plt.title('Edges in ROI')
plt.axis('off')
plt.show()
```

Mask



Edges in ROI



1.3 Fitting lines using Hough transform (15 points)

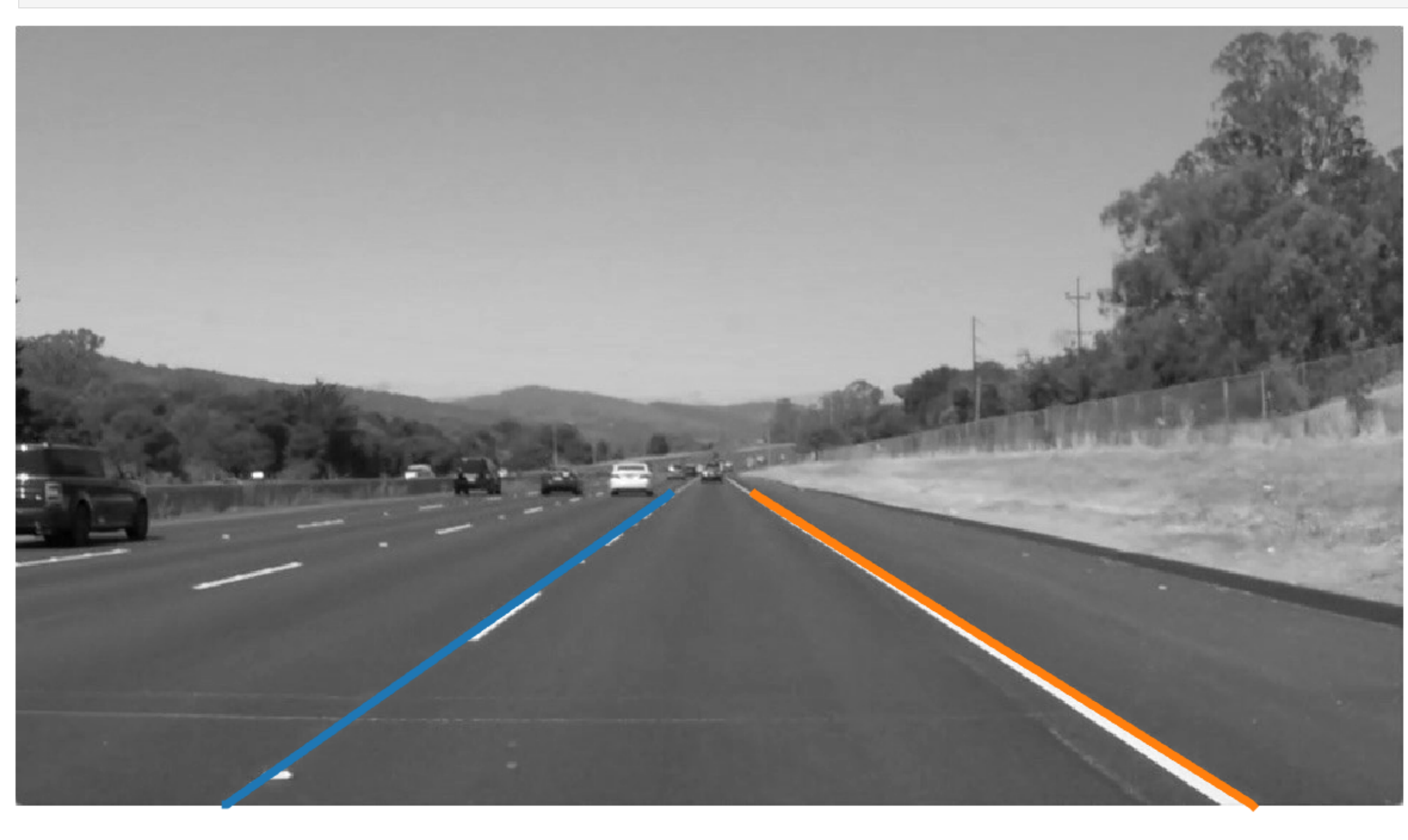
The output from the edge detector is still a collection of connected points. However, it would be more natural to represent a lane as a line parameterized as $y = ax + b$, with a slope a and y-intercept b. We will use Hough transform to find parameterized lines that represent the detected edges.

In general, a straight line $y = ax + b$ can be represented as a point (a, b) in the parameter space. This is the parameterization we often use when introducing the Hough transform. However, this cannot represent vertical lines as the slope parameter will be unbounded. Alternatively, we parameterize a line using $\theta \in [-\pi, \pi]$ and $\rho \in \mathbb{R}$ as follows:

$$\rho = x \cdot cos\theta + y \cdot sin\theta$$

Using this parameterization, we can map every point in xy-space to a sine-like line in $\theta\rho$-space (or Hough space). We then accumulate the parameterized points in the Hough space and choose points (in Hough space) with highest accumulated values. A point in Hough space then can be transformed back into a line in xy-space.

In [34]:
```
def hough_transform(img):
    """ Transform points in the input image into Hough space.

    Use the parameterization:
        rho = x * cos(theta) + y * sin(theta)
    to transform a point (x,y) to a sine-like function in Hough space.

    Args:
        img: binary image of shape (H, W).

    Returns:
        accumulator: numpy array of shape (m, n).
        rhos: numpy array of shape (m, ).
        thetas: numpy array of shape (n, ).
    """
    # Set rho and theta ranges
    W, H = img.shape
    diag_len = int(np.ceil(np.sqrt(W * W + H * H)))
    rhos = np.linspace(-diag_len, diag_len, diag_len * 2 + 1)
    thetas = np.deg2rad(np.arange(-90.0, 90.0))

    # Cache some reusable values
    cos_t = np.cos(thetas)
    sin_t = np.sin(thetas)
    num_thetas = len(thetas)

    # Initialize accumulator in the Hough space
    accumulator = np.zeros((2 * diag_len + 1, num_thetas), dtype=np.uint64)
    ys, xs = np.nonzero(img)

    # Transform each point (x, y) in image
    # Find rho corresponding to values in thetas
    # and increment the accumulator in the corresponding coordiate.
    ### YOUR CODE HERE

    # Only consider about edge points which are non-zero
    for i in range(ys.shape[0]):
        for t in range(num_thetas):
            rho = np.round(xs[i] * cos_t[t] + ys[i] * sin_t[t]).astype(int)
            r = rho + diag_len
            accumulator[r, t] += 1



    ### END YOUR CODE

    return accumulator, rhos, thetas
```

In [35]:
```
# Perform Hough transform on the ROI
acc, rhos, thetas = hough_transform(roi)

# Coordinates for right lane
xs_right = []
ys_right = []

# Coordinates for left lane
xs_left = []
ys_left = []

for i in range(20):
    idx = np.argmax(acc)
    r_idx = idx // acc.shape[1]
    t_idx = idx % acc.shape[1]
    acc[r_idx, t_idx] = 0 # Zero out the max value in accumulator

    rho = rhos[r_idx]
    theta = thetas[t_idx]

    # Transform a point in Hough space to a line in xy-space.
    a = - (np.cos(theta)/np.sin(theta)) # slope of the line
    b = (rho/np.sin(theta)) # y-intersect of the line

    # Break if both right and left lanes are detected
    if xs_right and xs_left:
        break

    if a < 0: # Left lane
        if xs_left:
            continue
        xs = xs_left
        ys = ys_left
    else: # Right Lane
        if xs_right:
            continue
        xs = xs_right
        ys = ys_right

    for x in range(img.shape[1]):
        y = a * x + b
        if y > img.shape[0] * 0.6 and y < img.shape[0]:
            xs.append(x)
            ys.append(int(round(y)))

plt.imshow(img)
plt.plot(xs_left, ys_left, linewidth=5.0)
plt.plot(xs_right, ys_right, linewidth=5.0)
plt.axis('off')
plt.show()
```